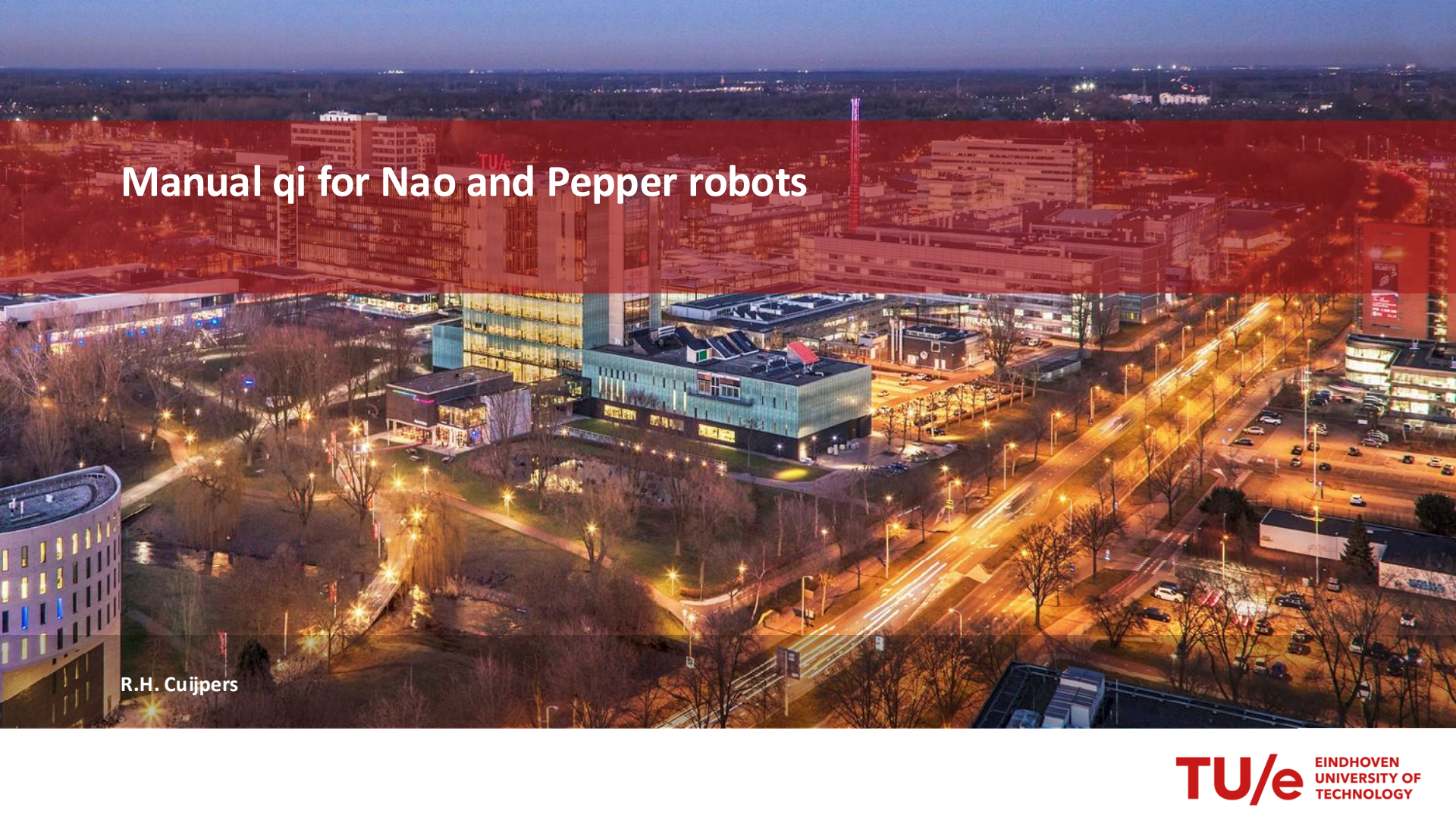


An aerial night photograph of the TU/e campus in Eindhoven, featuring modern buildings, a central green area with a bridge over a stream, and surrounding city lights. A semi-transparent red rectangle is overlaid on the top half of the image.

Manual qi for Nao and Pepper robots

R.H. Cuijpers

Connect to robot using qi

After installing qi, you can connect to the robot like this:

```
import qi

#robot_ip="192.168.0.111"
robot_ip="127.0.0.1"
robot_port=9559

def connect_robot(robot_ip, robot_port):
    print("connecting to robot " + "tcp://" + robot_ip + ":" + str(robot_port))
    return qi.Application(url= "tcp://" + robot_ip + ":" + str(robot_port))
```

And create a session:

```
if __name__ == "__main__":
    app = connect_robot(robot_ip, robot_port)
    app.start()
    session = app.session
```

Landmark detection

1. Initialize landmark detection

```
landmark_detection = session.service("ALLandMarkDetection")  
landmark_detection.subscribe("LandmarkDetector", 500, 0.0)
```

2. After that the Landmark can be detected using (see next slide):

```
memproxy= session.service("ALMemory")  
data = memory_proxy.getData("LandmarkDetected")
```

- Data contains many fields. You can extract the markerinfo as shown on the next slide. It contains 6 fields for each detected landmark, for example the first detected landmark contains:

markerinfo[0][0]	#markerID
markerinfo[0][1]	#alpha - x location in camera angle
markerinfo[0][2]	#beta - y location
markerinfo[0][3]	#sizeX
markerinfo[0][4]	#sizeY
markerinfo[0][5]	#orientation about vertical w.r. Nao's head

Landmark detection

```
def DetectLandMark(memory_proxy):
    data = memory_proxy.getData("LandmarkDetected")
    if data==None:
        data=[] # otherwise the next statement fails ...
    if len(data)==0:
        detected=False
        timestamp=time()
        markerInfo=[]
    else:
        detected=True
    #timestamp=data[0][0]+1E-6*data[0][1] #this works but only if a landmark is detected
    timestamp=time()
    markerInfo=[]
    for p in data[1]:
        markerInfo.append([p[1][0], #markerID
                           p[0][1], #alpha - x location in camera angle
                           p[0][2], #beta - y location
                           p[0][3], #sizeX
                           p[0][4], #sizeY
                           p[0][5] #orientation about vertical w.r. Nao's head
                           ])
    return detected, timestamp, markerInfo
```

Landmark detection

More elegant is to use a callback function.

First subscribe to the “LandmarDetected” events:

```
memory = session.service("ALMemory")  
# Connect the event callback.  
subscriber = memory.subscriber("LandmarkDetected")  
subscriber.signal.connect(on_landmark_detected)
```

And define the callback function (next slide)

Landmark detection

```
def on_landmark_detected(value):  
    """  
    Callback for event LandmarkDetected.  
    """  
  
    if value == []: # empty value when the landmark disappears  
        return  
  
    else:  
        # First Field = TimeStamp.  
        timeStamp = value[0]  
        print("Time Stamp is: " + str(timeStamp))  
  
        # Second Field = array of mark_Info's.  
        markInfoArray = value[1]  
        for markInfo in markInfoArray:  
            # First Field = Shape info.  
            markShapeInfo = markInfo[0]  
  
            # Second Field = Extra info (ie, mark ID).  
            markExtraInfo = markInfo[1]  
            print("mark ID: %d" % (markExtraInfo[0]))  
            print(" alpha %.3f - beta %.3f" % (markShapeInfo[1], markShapeInfo[2]))  
            print(" width %.3f - height %.3f" % (markShapeInfo[3], markShapeInfo[4]))
```